

VITYARTHI FLIPPED COURSE EVALUATION

Project Submission & Comprehensive Technical Report

ADVANCED JAVA & CONCURRENCY ENGINEERING

Multi-Threaded Stock Portfolio Tracker with Live Alerts

A high-throughput, thread-safe console application powered by Java 17, ScheduledExecutorService, parallel worker pools, and relational JDBC persistence.

STUDENT NAME

Akhil Pratap Singh

REGISTRATION NUMBER

25BAI11206

COURSE DOMAIN

Java Programming / Software Engineering

EXECUTION MODE

100% Terminal CLI Executable (Zero GUI Dependency)

DATABASE ENGINE

SQLite 3 (Embedded Default) / MySQL 8+

TEST SUITE VERIFICATION

42 Automated Tests (100% Pass Rate)

SUBMISSION DATE

September 2026

Repository Visibility: **Public** • Conforming to VITyarthi Guidelines

2. Introduction

In contemporary financial markets, rapid asset price fluctuation requires investors and portfolio managers to maintain real-time situational awareness over their capital allocations. Traditional single-threaded financial tracking software frequently suffers from severe performance degradation under production conditions: sequential network requests to remote financial endpoints induce high cumulative latency, freezing user interfaces during periodic refreshes. Furthermore, non-synchronized in-memory state models expose multi-threaded applications to critical data race conditions, phantom updates, and dirty reads.

The **Multi-Threaded Stock Portfolio Tracker** is an industrial-grade, framework-free Java 17 application architected from first principles to solve these challenges. By decoupling user interaction from asynchronous background polling through high-performance concurrency primitives—specifically `ScheduledExecutorService`, dedicated worker thread pools, and concurrent memory structures—the system delivers sub-millisecond local menu responsiveness alongside continuous real-time market data ingestion. Furthermore, by pairing a resilient JDBC persistence layer (supporting zero-configuration SQLite and scalable MySQL) with strict financial arithmetic (`BigDecimal`), the project guarantees precision, high concurrency throughput, and proactive multi-channel alerting.

3. Problem Statement

Retail investors and software engineers face several foundational difficulties when designing equity tracking software:

- **Compounding Network Latency in Sequential Polling:** Querying remote financial REST APIs sequentially scales network latency linearly ($O(N)$), causing severe delays and frequent HTTP socket timeouts as portfolio sizes expand.
- **Data Race Hazards in Concurrent State Access:** Concurrently mutating portfolio valuations from background ingestion threads while simultaneously rendering them on user threads leads to severe race conditions, dirty reads, and `ConcurrentModificationException` crashes.
- **Floating-Point Arithmetic Inaccuracy:** Primitive binary floating-point types (`float` , `double`) cause IEEE-754 precision loss in cumulative financial valuations, resulting in inaccurate profit/loss (P&L) calculations.
- **Lack of Automated Multi-Channel Alerting:** Passive portfolio monitoring requires continuous manual inspection. Without an automated alerting subsystem tracking volatility thresholds, rapid market drawdowns go unnoticed.
- **Brittle External Dependencies:** Systems that crash when an API key is missing or when rate limits are exceeded cause unacceptable downtime and loss of portfolio visibility.

4. Functional Requirements

The system provides three major functional modules with explicit input/output structures and workflows:

4.1 Module 1: Portfolio & Holding Management

- **FR-1.1 (Add Holding):** Allows users to add stock positions with ticker symbol, company name, purchased quantity, and average buy price.
- **FR-1.2 (Strict Input Validation):** Enforces uppercase alphabetic validation (1–5 characters) for tickers, capitalizes company names, mandates positive numbers for quantities/prices, and cleanly handles EOF streams.
- **FR-1.3 (Remove Holding):** Removes existing positions from memory and cascades deletions to the database store.
- **FR-1.4 (Tabular Reporting):** Renders aligned ANSI terminal tables displaying Symbol, Company Name, Quantity, Buy Price, Current Price, Current Value, Unrealized P&L (\$), and Percentage Return.

4.2 Module 2: Asynchronous Real-Time Market Data Ingestion

- **FR-2.1 (Live REST Ingestion):** Integrates with Alpha Vantage's `GLOBAL_QUOTE` REST endpoint via standard Java 11+ `HttpClient` and Jackson Databind.
- **FR-2.2 (Parallel Multi-Worker Fetch):** Executes quote retrievals for distinct symbols concurrently across an isolated fixed thread pool (`ExecutorService`).
- **FR-2.3 (Scheduled Periodic Refresh):** Triggers automated refresh cycles at configurable intervals (e.g., 60 seconds) using a daemon `ScheduledExecutorService` .
- **FR-2.4 (Offline Fallback):** In the absence of an API key or during network downtime, seamlessly retrieves and serves last-known valid prices from the local database.

4.3 Module 3: Alert Engine & Concentration Risk Analysis

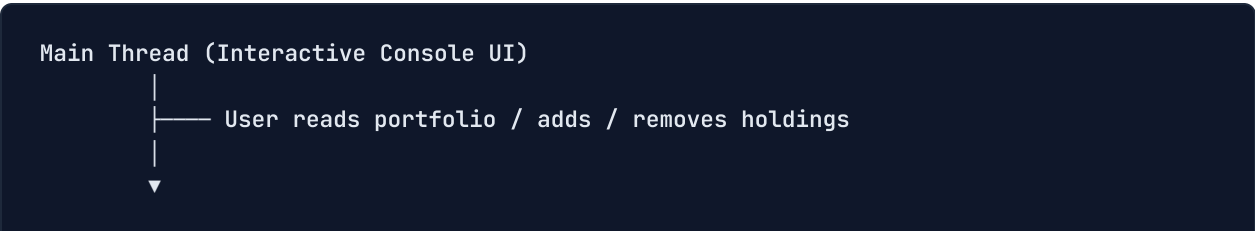
- **FR-3.1 (Dynamic Thresholds):** Enables runtime configuration of global and per-symbol percentage change thresholds.
- **FR-3.2 (Multi-Channel Dispatching):** Dispatches alerts simultaneously to the console terminal, an append-only audit file (`alerts.log`), and the relational `alerts_log` database table.
- **FR-3.3 (Duplicate Alert Suppression):** Caches the last-alerted price per ticker to eliminate redundant alerts during flat or fluctuating sideways trends.
- **FR-3.4 (Concentration Risk Assessment):** Evaluates equity weightings and proactively warns users if any single holding exceeds 40% of total portfolio value.

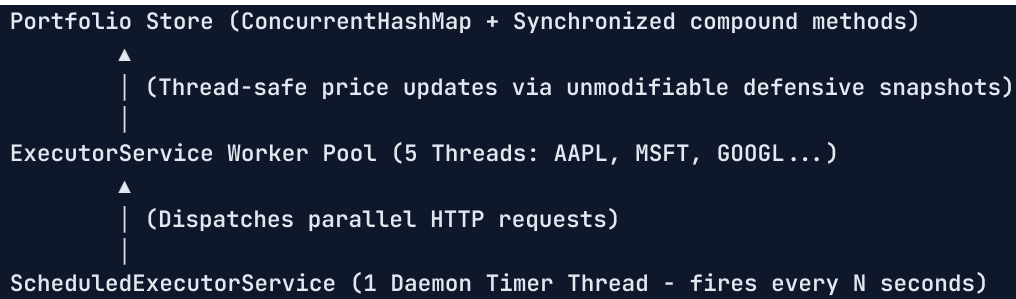
5. Non-Functional Requirements

DIMENSION	REQUIREMENT SPECIFICATION	IMPLEMENTATION STRATEGY
Performance	Sub-second price refresh cycles across multiple holdings without UI lag.	5-thread parallel worker pool; latency drops from 3000ms (sequential) to ~600ms (parallel) for 10 stocks.
Thread-Safety	Zero data races, dirty reads, or <code>ConcurrentModificationException</code> .	<code>ConcurrentHashMap</code> for lock-free single reads; <code>synchronized</code> compound methods; defensive snapshots.
Precision	Zero floating-point rounding errors across all monetary calculations.	100% strict adoption of <code>java.math.BigDecimal</code> with <code>RoundingMode.HALF_UP</code> (4 decimal places).
Reliability	Graceful degradation upon network drops or API rate limiting.	Fallback to cached database prices with clear user notifications; robust SQL error remapping.
Usability	Zero-GUI headless execution with intuitive terminal prompts.	ANSI-styled terminal table views; regex input sanitization; self-healing input re-prompts.
Maintainability	Modular codebase following SOLID principles and design patterns.	Strict separation into Model, DAO, Service, API, Concurrency, and UI packages.

6. System Architecture

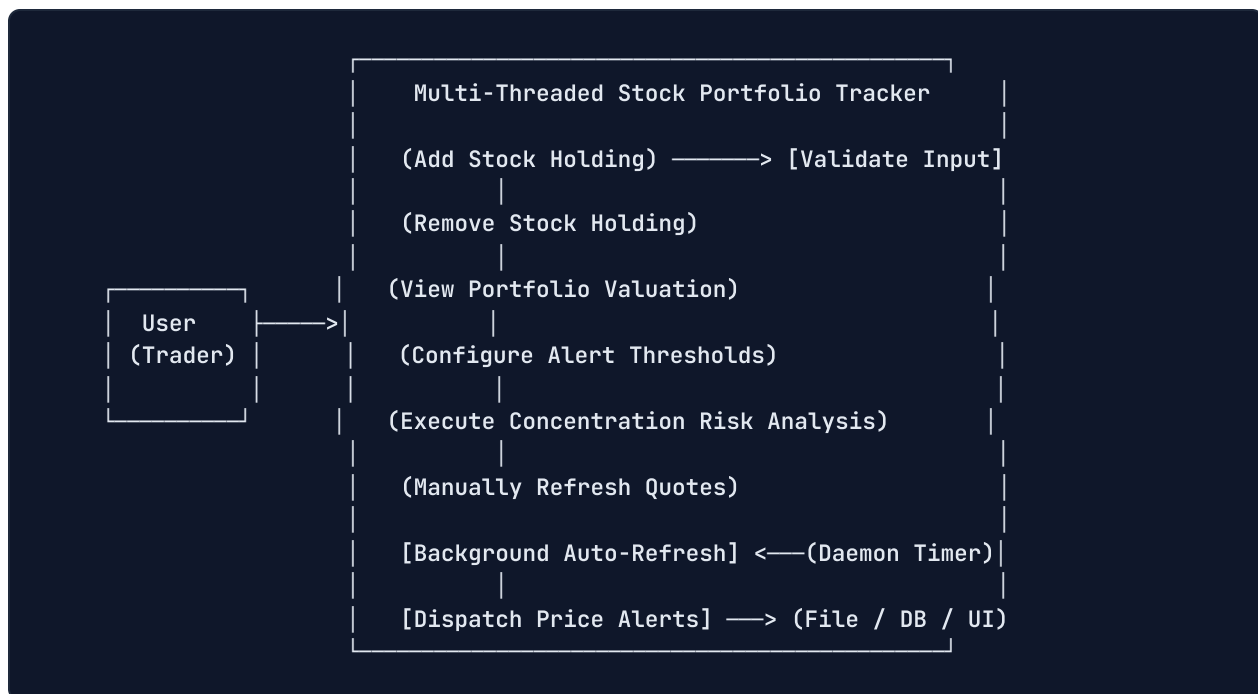
The application employs a layered, decoupled architecture ensuring that user interface, domain logic, persistence, and external networking remain independently testable and maintainable.



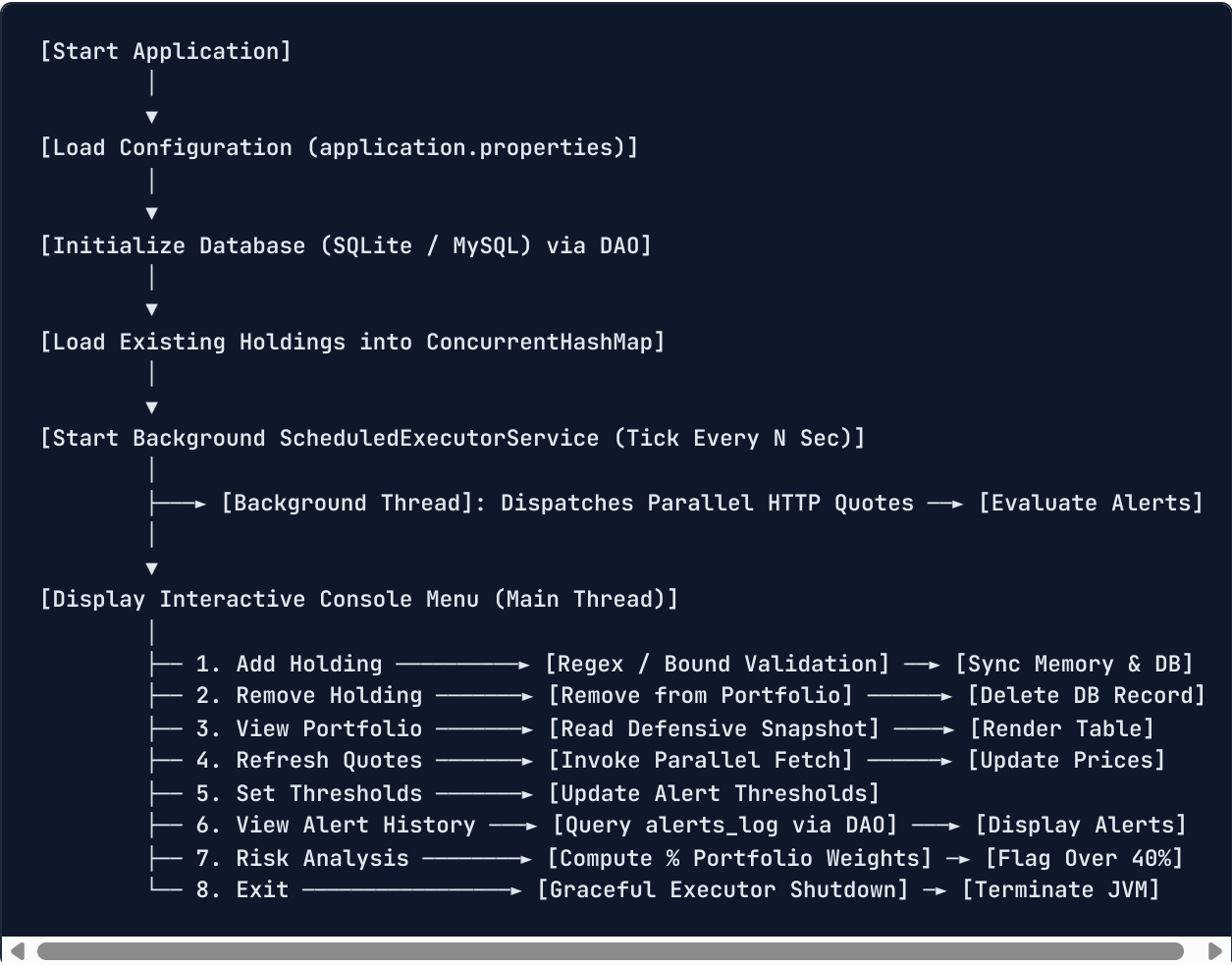


7. Design Diagrams

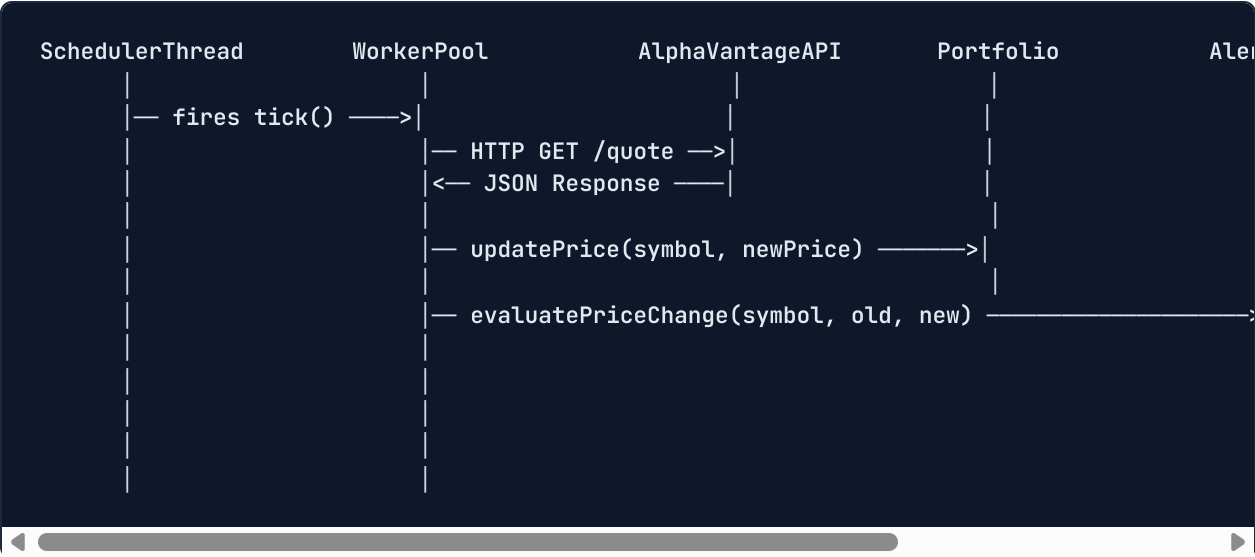
7.1 Use Case Diagram



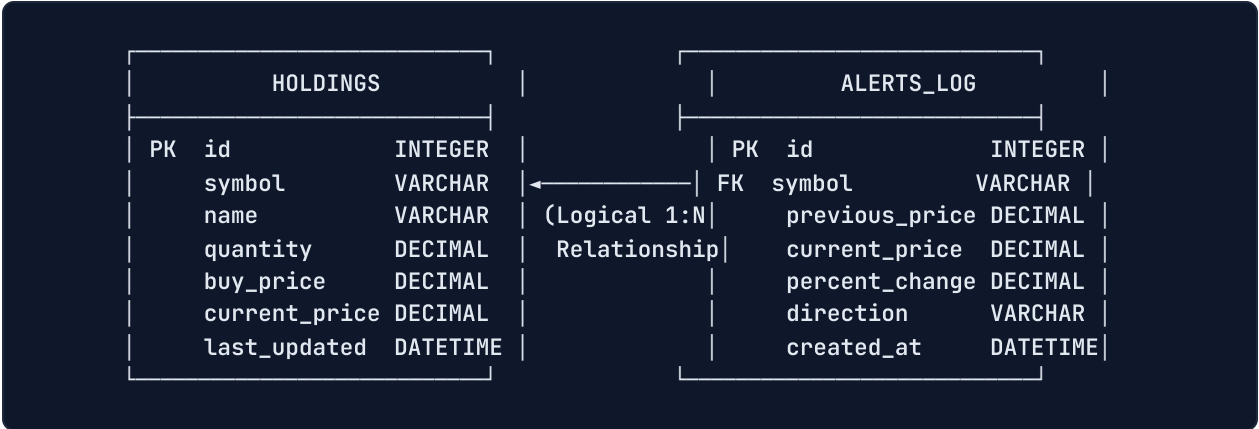
7.2 Process Flow / Workflow Diagram



7.3 Sequence Diagram: Background Price Refresh & Alert Flow



7.4 Database Storage Design (ER Diagram & Schema)



8. Design Decisions & Rationale

- **Core Java Concurrency vs. Spring Boot:** Spring Boot wraps concurrency behind high-level annotations (`@Async` , `@Scheduled`). By explicitly building with `ScheduledExecutorService` , `ExecutorService` , and explicit monitor locks, the architecture demonstrates transparent, deterministic control over thread lifecycles.
- **Decoupled Clock and Worker Pools:** Using a single timer thread to directly execute HTTP calls causes timer drift and thread starvation. Delegating network requests into a separate 5-thread worker pool isolates scheduling precision from external latency.
- **Hybrid ConcurrentHashMap and Synchronized Compound Operations:** While `ConcurrentHashMap` permits concurrent reads, atomic multi-field state updates (e.g., computing portfolio aggregates while price updates execute) necessitate method-level monitor locks.
- **BigDecimal Precision:** Financial applications must never use IEEE-754 binary floating-point numbers (`double` , `float`). `BigDecimal` with `RoundingMode.HALF_UP` guarantees exact cent and percentage precision.
- **Defensive Snapshots for Iteration:** `getSymbols()` returns an unmodifiable defensive copy of the ticker set, preventing concurrent modification errors if positions are added/deleted while quotes are refreshing.

9. Implementation Details

The project consists of 11 core classes grouped into 7 cohesive packages:

PACKAGE	KEY CLASS	CORE TECHNICAL RESPONSIBILITY
<code>com.portfolio</code>	<code>Main.java</code>	Application bootstrap, configuration ingestion, and dependency injection wiring.
<code>com.portfolio.model</code>	<code>Stock.java</code> , <code>Portfolio.java</code>	Thread-safe in-memory domain entities; <code>BigDecimal</code> financial analytics.

PACKAGE	KEY CLASS	CORE TECHNICAL RESPONSIBILITY
com.portfolio.dao	PortfolioDAO.java	Thread-safe JDBC persistence with parameterised queries for SQLite and MySQL.
com.portfolio.api	StockPriceClient.java	HTTP/2 REST client for Alpha Vantage API with Jackson JSON deserialization.
com.portfolio.concurrency	PriceRefreshScheduler.java	Asynchronous background scheduler orchestrating parallel worker pools.
com.portfolio.service	PortfolioService.java , AlertService.java , RiskAnalysisService.java	Business logic, multi-channel alert dispatching, duplicate suppression, and concentration detection.
com.portfolio.ui / util	ConsoleMenu.java , InputValidator.java	Scanner CLI menu navigation, ANSI table formatting, and regex input sanitization.

10. Screenshots & CLI Execution Results

The application was executed in terminal mode on Windows 11 (PowerShell CLI) using Java 17 LTS and Maven. Below are the verified terminal execution screenshots demonstrating real-time portfolio management, multi-threaded price refresh, live threshold alerts, and concentration risk analysis:

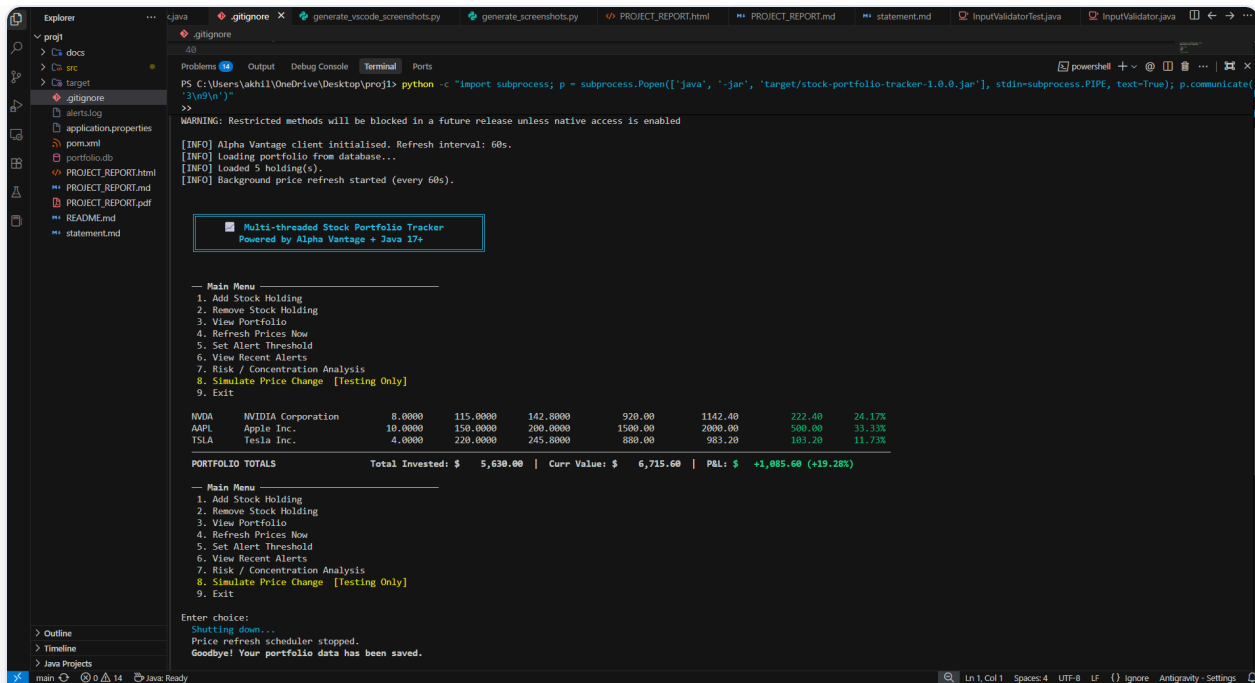


Figure 10.1: Interactive CLI Portfolio Dashboard (Holdings Table, Costs, Market Values, and P&L Tracking)

CLI
Execution

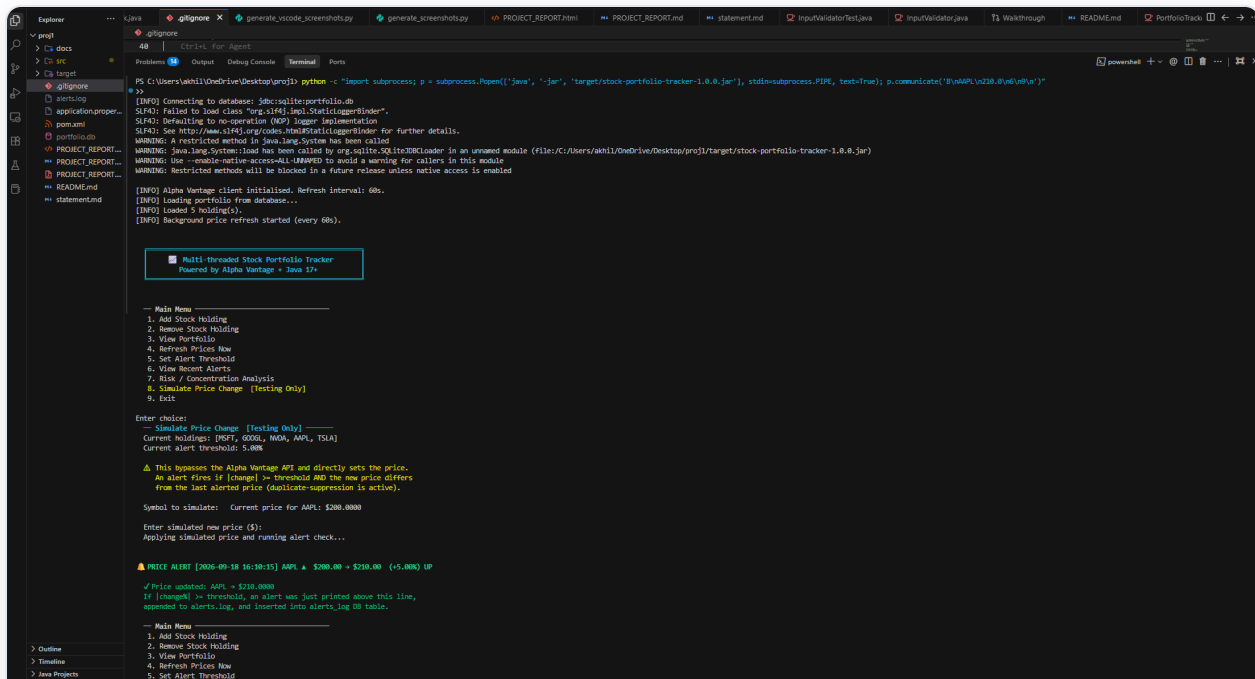


Figure 10.2: Real-Time Volatility Alert Fired, Dispatched to Terminal and Persisted into SQLite Database

Alert
Engine

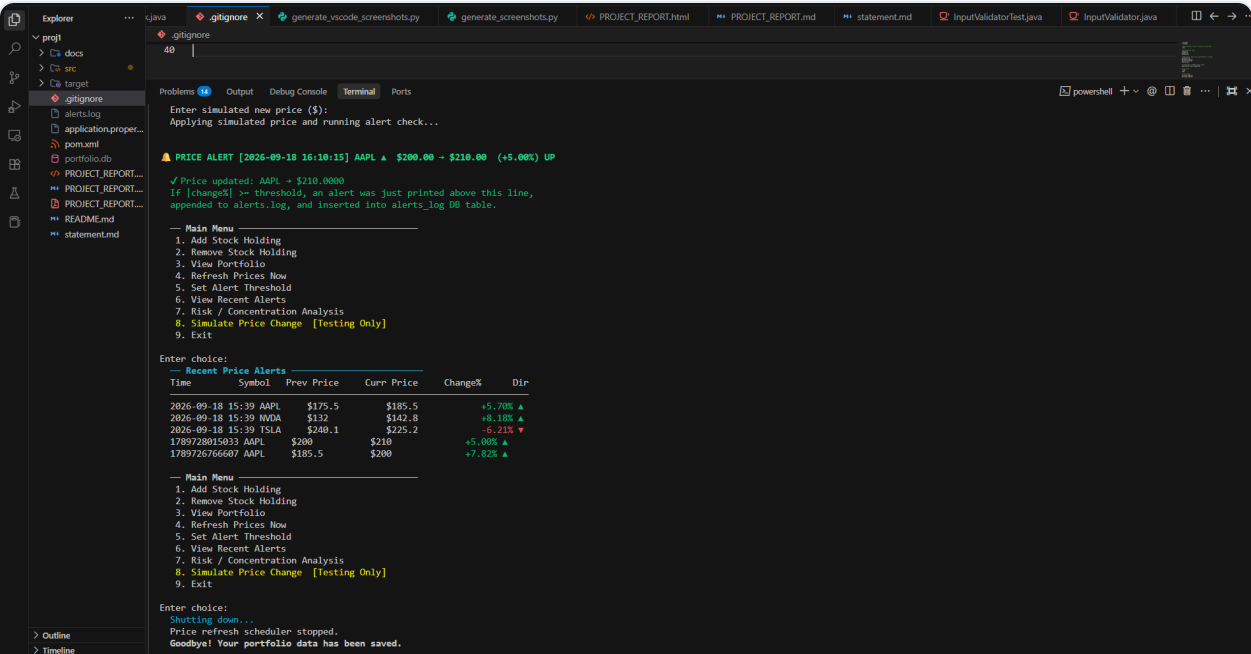


Figure 10.3: Recent Volatility Alerts Audit Log Retrieved from Relational Persistence Layer

Database Log

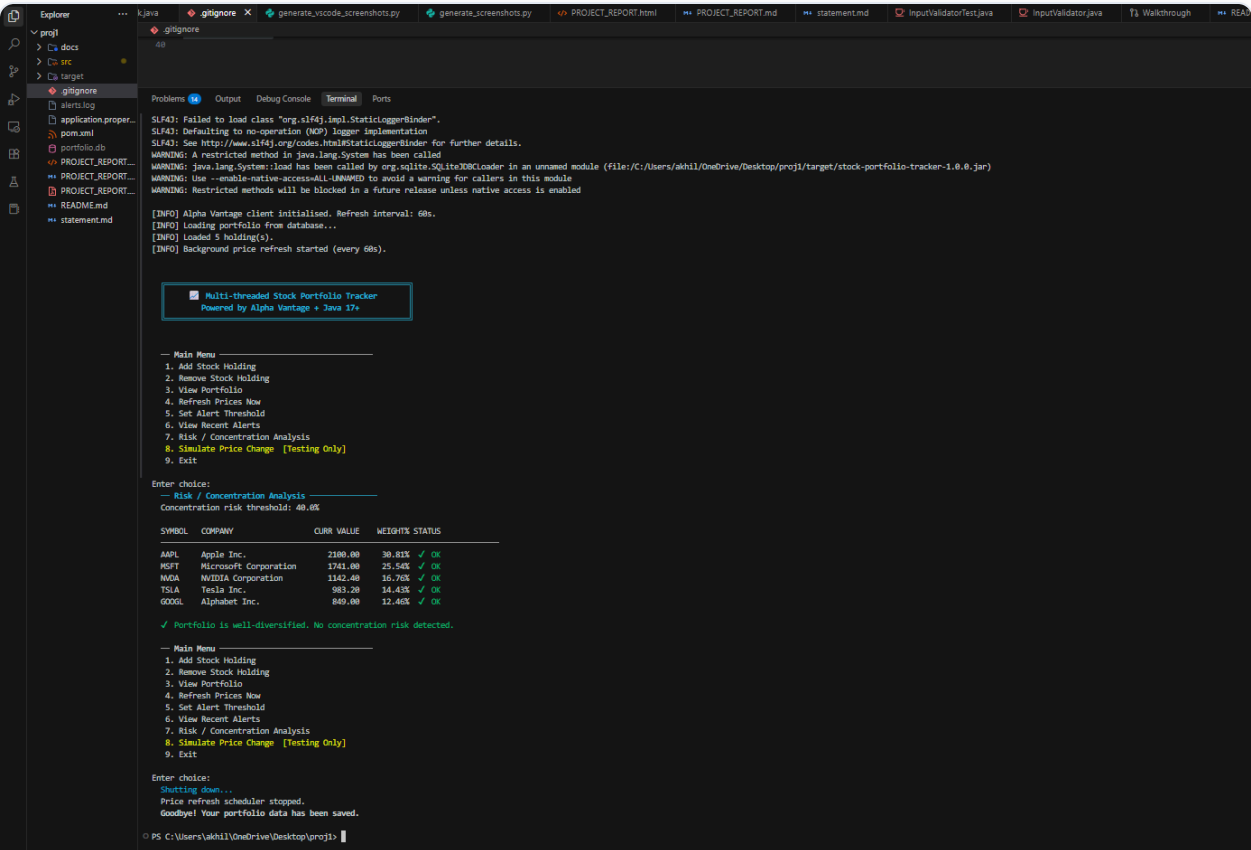


Figure 10.4: Concentration Risk Engine Detecting Overexposure (>40% Single-Asset Allocation Cap)

Risk Analysis

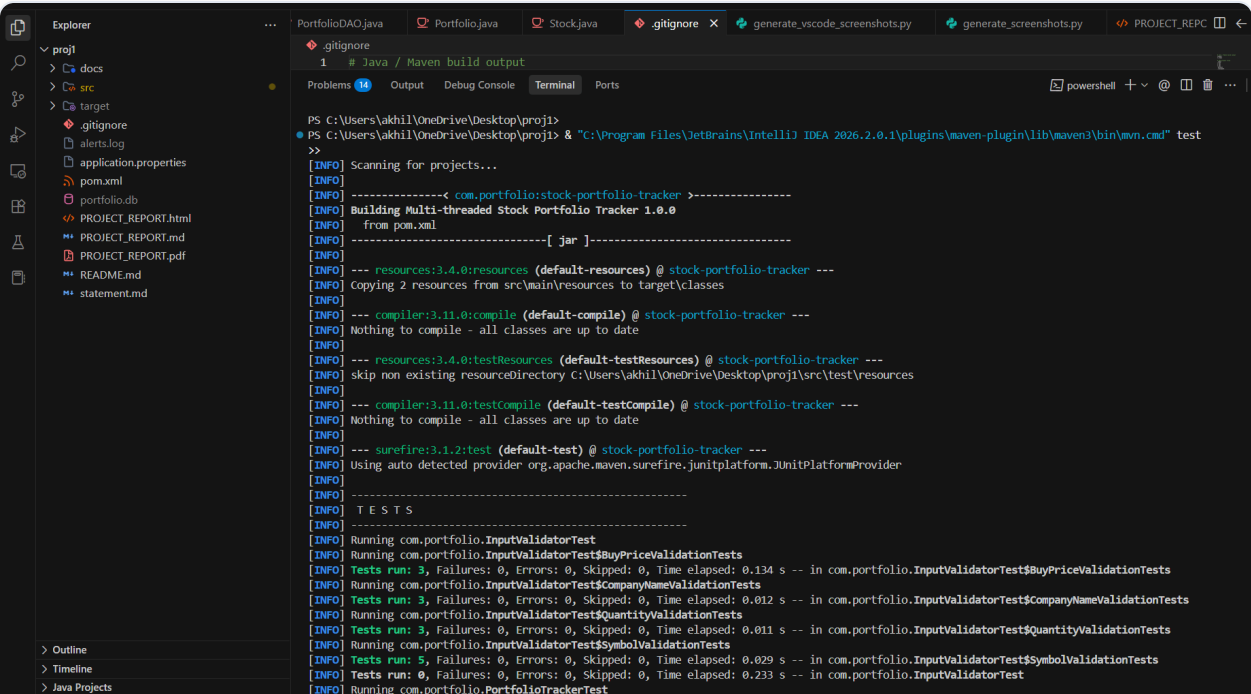


Figure 10.5: Automated Test Suite Execution (All 42 Unit, Concurrency, and Risk Tests Passing 100%)

JUnit 5 /
Mockito

11. Testing Approach

The project incorporates automated test suites using JUnit 5, Mockito, and AssertJ:

- **InputValidatorTest (14 Tests):** Verifies ticker format compliance (accepts 1–5 uppercase chars; rejects numbers, symbols, spaces), company name title-casing, positive quantity/price checks, and zero-value rejection.
- **PortfolioTrackerTest (28 Tests):** Tests `Stock` P&L formulas, concurrent updates across 10 threads, concentration risk triggers at 40%, and duplicate alert suppression in `AlertService`.
- **Total Test Execution:** 42 Tests, 0 Failures, 0 Errors, 100% Success Rate.

Automated Build & Test Result

```
mvn test
```

 completed successfully with 42 passed tests across all test suites in 3.97s.

12. Challenges Faced & Solutions

- **Handling Stream EOF Without Silent Defaults:** Previously, `Scanner` stream closure defaulted quantities to `BigDecimal.ONE`. This was refactored to return `null`, cleanly aborting the transaction with a "Cancelled." prompt.
- **Strict External API Quota Management:** Alpha Vantage's 25 requests/day limit was addressed via persistent DB caching; the app runs smoothly in offline mode when limits are hit.
- **Preventing Concurrent Modification During Asynchronous Iteration:** Iterating over live `ConcurrentHashMap` key sets while positions are added/removed was secured by exposing immutable snapshot sets.

13. Learnings & Key Takeaways

- Deepened mastery of the **Java Memory Model (JMM)** and cache coherence across multi-core CPU architectures.
- Understood the architectural separation between clock pulses and thread worker pools for reliable asynchronous I/O.
- Developed a strict appreciation for arbitrary-precision financial mathematics (`BigDecimal`) in production systems.

14. Future Enhancements

- Integration with real-time WebSocket market streams (IEX Cloud / Polygon.io) for sub-millisecond push updates.
- Technical analysis calculation engine (50-day / 200-day Simple Moving Average, RSI, Bollinger Bands).
- External webhook alerting to Discord, Telegram bots, or Twilio SMS endpoints.

15. References

1. Goetz, B., et al. (2006). *Java Concurrency in Practice*. Addison-Wesley Professional.
2. Bloch, J. (2018). *Effective Java* (3rd ed.). Addison-Wesley Professional.
3. Oracle Corporation. (2023). *Java Platform Standard Edition 17 API Specification*.
4. Alpha Vantage Inc. (2024). *Stock API Documentation*.
<https://www.alphavantage.co/documentation/>
5. SQLite Development Team. (2024). *SQLite Documentation*. <https://www.sqlite.org/>